

TD9 : Protéger une application web en appliquant un codage sécurisé

Délio Duchesne-Brajon
Groupe 1

[TD9 : Protéger une application web en appliquant un codage sécurisé](#)

[Introduction :](#)

[TP :](#)

Introduction :

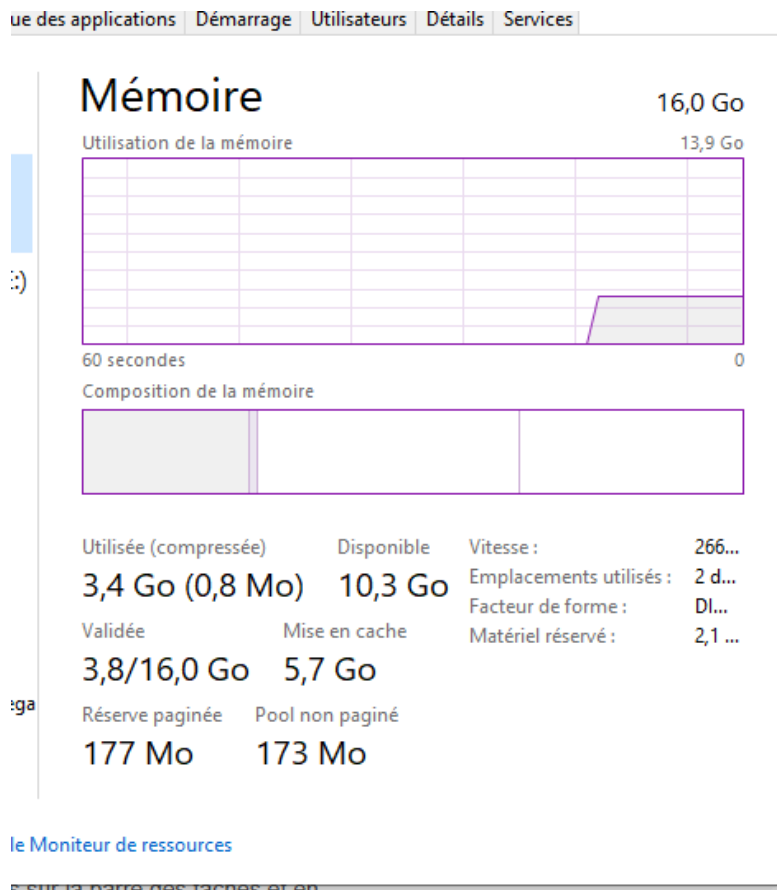
L'objectif de ce TP était de découvrir les principales failles de sécurité présentes dans une application web et de comprendre comment les corriger grâce à un codage sécurisé.

Pour cela, plusieurs machines virtuelles ont été configurées puis utilisées avec l'application Mutillidae afin de réaliser différentes manipulations, notamment des injections SQL.

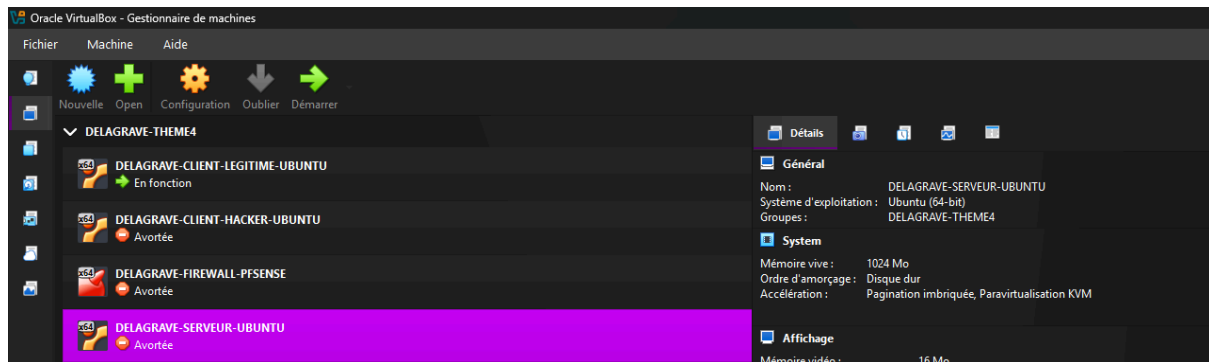
Ce TP a permis de comprendre l'importance de la validation des données utilisateur et des mécanismes de protection mis en place pour sécuriser une application web.

TP :

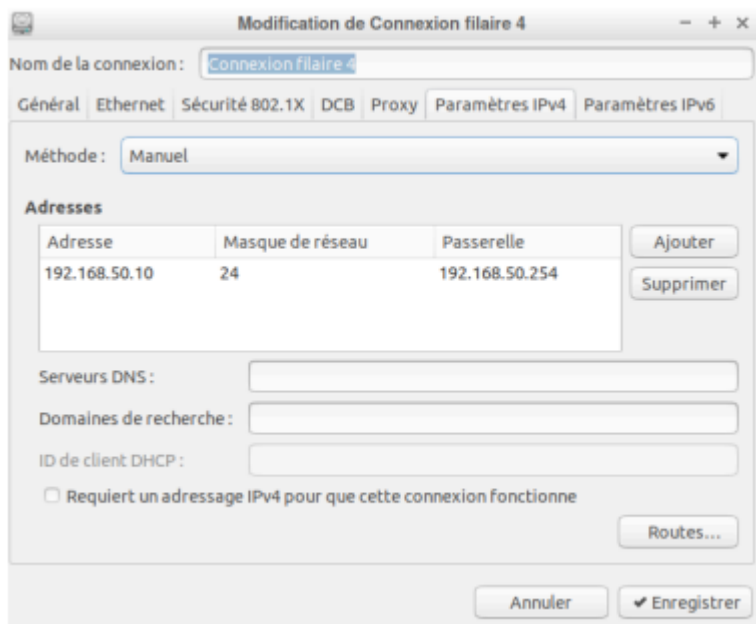
Dans un premier temps il fallu vérifier si le pc était apte à être configuré et donc marcher pour le TP soit vérifier s'il y avait 6Go de mémoire vive disponible pour faire tourner les 4 machines virtuelles en mêmes temps



Ensuite téléchargez donc le fichier avec les 4 VM et les mettre dans le logiciel :



Par la suite le vrai début du TP, il fallait suivre à l'intérieur même d'une des VM les instructions d'une vidéo s'y trouvant qui nous demandait de configurer les réseau de chaque VM :



Dans cette partie du TP, il fallait réaliser une injection SQL sur la page Mutillidae user-info.php avec un niveau de sécurité réglé à 0, c'est-à-dire sans aucune protection activée.

Pour cela, une injection SQL a été saisie dans le champ de recherche :

```
' OR '1'='1      ou bien :      ' OR 1=1--
```

Une fois la requête exécutée, il a été possible de constater que l'application ne réalisait aucun contrôle sur les données saisies par l'utilisateur. La page affichait alors la liste complète des utilisateurs enregistrés dans la base de données avec leurs identifiants, noms d'utilisateur et mots de passe.

Cette manipulation montre qu'en absence de validation des entrées utilisateur, il est possible de modifier la logique d'une requête SQL afin de contourner le système d'authentification et d'accéder à des données sensibles.

Par la suite, le niveau de sécurité de Mutillidae a été augmenté à 5 grâce au bouton « Toggle Security » afin d'activer les protections de sécurité avancées.

Une fois le niveau de sécurité modifié, la même injection SQL a été reproduite. Cette fois-ci, l'attaque ne fonctionnait plus. L'application n'affichait plus les informations de la base de données et retournait soit une erreur, soit aucun résultat.

Cela est dû aux différents mécanismes de protection activés côté serveur :

- vérification et validation des données saisies ;

- filtrage des caractères dangereux ;
- limitation de certaines entrées utilisateur ;
- encodage des données affichées.

Enfin, une comparaison du fichier `user-info.php` entre le niveau de sécurité 0 et le niveau 5 a permis d'observer les différences de configuration entre une application vulnérable et une application sécurisée.

Dans la version non sécurisée, aucune validation n'était activée :

```
$IEncodeOutput = FALSE;  
$IEnableHTMLControls = FALSE;  
$IJavaScriptValidation = FALSE;
```

Alors que dans la version sécurisée, plusieurs protections étaient activées :

```
$IEncodeOutput = TRUE;  
$IEnableHTMLControls = TRUE;  
$IJavaScriptValidation = TRUE;
```

Cette étape du TP a permis de comprendre l'importance du codage sécurisé dans le développement d'applications web.

Elle montre notamment qu'il est essentiel de toujours contrôler les données envoyées par les utilisateurs, d'utiliser des requêtes préparées pour les accès à la base de données et d'encoder correctement les données affichées afin de limiter les risques d'attaques informatiques.